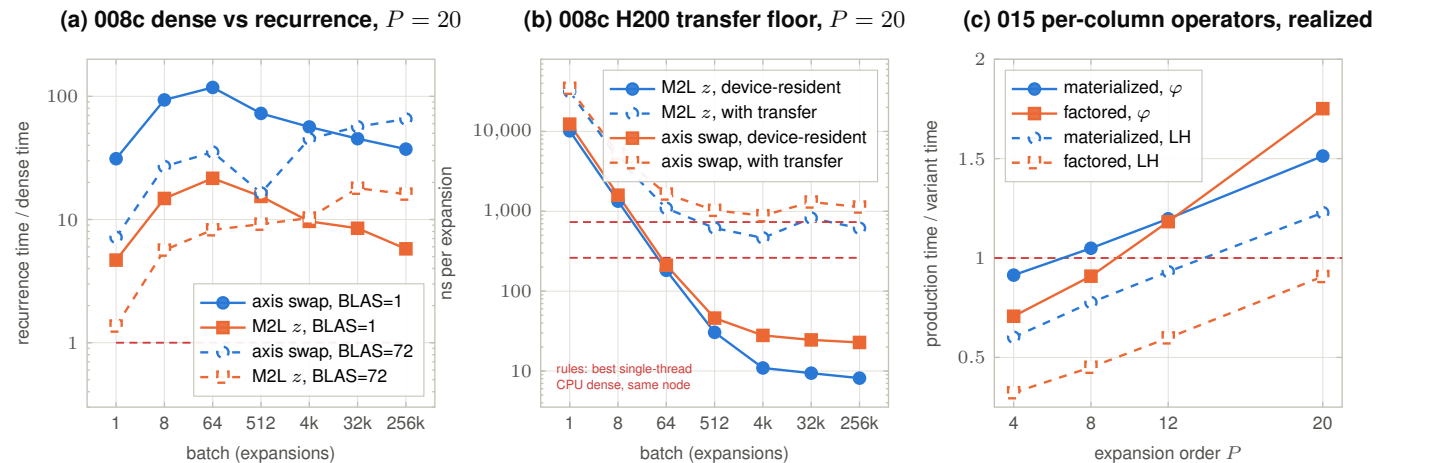


Fig. 7 — Provenance: the 008c projections, the transfer floor that forced device residency, and the projection the CPU operators did not realize



Machines / regimes: (a) m13h-1-1 Xeon Platinum 8568Y+ with OpenBLAS pinned to 1 thread, and m12-2-5 EPYC 7763 with 72 BLAS threads; both BLAS installs tuned. (b) m13h-1-1 NVIDIA H200, hand-written fused scalar CUDA kernel — deliberately *not* cuBLAS. (c) tmpfac-126-17, Apple M2 laptop, Julia 1.12.5, BLAS=1, batch 4096. All Float64. **Data:** impl_performance_baseline/{m13h-1-1/dense_vs_loop_blas1.csv, m12-2-5/dense_vs_loop_blas72.csv, m13h-1-1/dense_gpu.csv}, axis_swap/tmpfac-126-17.et.byu.edu/m2l_variants_blas1.csv (tables fig07a_*, fig07b_*, fig07c_*). **Reads as:** this is the honest arc of the project. Task 008c projected 31–118 $\times$ for a dense axis swap over the recurrence, with multi-thread BLAS *losing* below batch ≈ 8 (a). It also showed that per-expansion host–device transfer floors out worse than the best CPU dense at every batch width, so keeping expansions device-resident was an architectural requirement rather than an optimisation (b). Task 015 then measured what the per-column operators of task 014 actually delivered on a CPU: at most 1.75 $\times$ (factored, φ -only, $P = 20$) and below 1 $\times$ for Lamb–Helmholtz almost everywhere (c). The projection in (a) was only realized later, by the whole-slab dense and fused-kernel GPU paths of tasks 019–024 — not by the per-column operators. **Scope:** every panel here is an *isolated* operator or stage microbenchmark — no panel in this figure runs an `fmm!` call, and none includes nearfield direct interactions. Contrast Figs. 3, 5 and 6, whose timings are complete end-to-end evaluations. **Caveats:** in (a) the large-batch recurrence rows are projections — only $\min(\text{batch}, \text{cap})$ expansions were timed and scaled (scaled_from_per_expansion=true). Panel (c) is a single-laptop measurement on a different machine, scope, and normalization from every other figure here, so it is shown as a ratio only and its microseconds must never be compared against the stage seconds of Fig. 3 or the step times of Figs. 5–6. Panel (b)’s `per_block_launch` variant is omitted (catastrophic at small batch); Float32 was measured and is not the lever — 1.2–1.27 $\times$ at large batch and slower below.